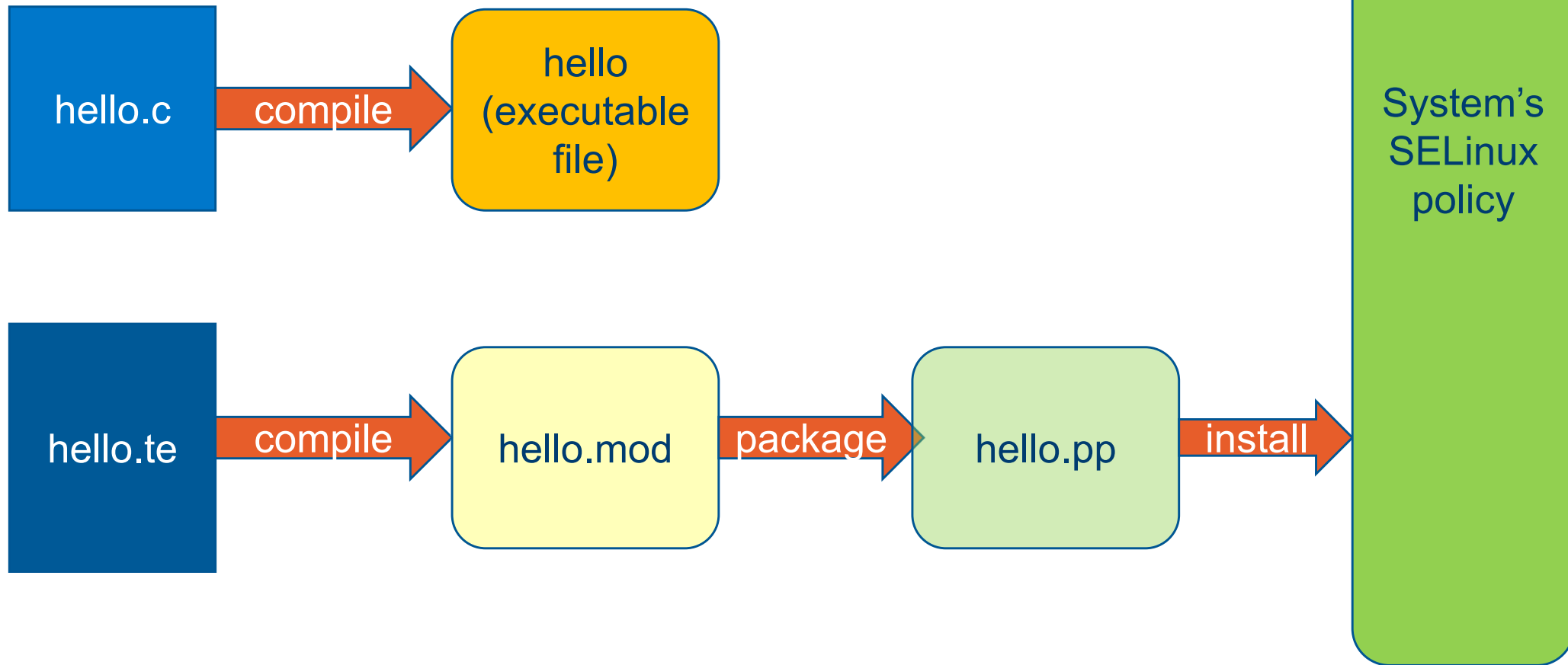


SELinux Policy Writing Tutorial

- Tips as we go through writing a simple policy for two examples:
 - Hello world example
 - Hello secret example

The files involved



Compiling, Installing, and Removing Policy Modules

- SELinux makefile doesn't exist anymore, so instead use these commands to compile and install your policy:

```
checkmodule -M -m mymodule.te -o mymodule.mod  
semodule_package -o mymodule.pp -m mymodule.mod  
semodule -i mymodule.pp (as root)
```

- If you later want to remove the module (to reload it), you should remove it with:

```
semodule -r mymodule (as root)
```

*** You will be using these commands a **lot** over the next few weeks. Consider making a bash script!!

Hello World example: Activity---getting started (do this on your own before we continue)

1. Download hello.c and hellosecret.c from Canvas
2. Compile them both --- see comments at top for how to do this. Don't panic if you're not familiar with C --- we will go over it.
3. Download your 'starter' hello.te file. You can also find this starter policy in Week 9's lecture slides.
4. Create your bash script for compiling (refer back to the "Compiling, Installing, and Removing Policy Modules" slide).

If you miss my demo (and are reading these slides outside of class), I recommend:

If you missed my demo and description of everything, the next best thing I can recommend is this reference:

<http://billauer.co.il/selinux-policy-module-howto.html#SECTION00060000000000000000> (focusing on the “Writing an SELinux Module” section).

Important notes:

- You need to add “role unconfined_r” to this example’s require clause
- The build process mentioned in the above link doesn’t work---use the process from earlier in this slide deck
- Since SELinux changes over time, beware that some of the things mentioned in the above link are no longer the case! The process is still valid though.

Step 1: Understand your program

What does it do?

Let's talk about our example .c programs now.



Step 2: Consider what permissions the program *should* have

These are the permissions you, as the policy writer, need to give it (by writing them into the policy at the start).

Ask yourself:

- What permissions should our program have?
- If there are any permissions you want to give your program: how do we ensure they are minimal permissions?
 - Consider that if you use existing types, there are other objects on the system that have been assigned them!



Step 3: Create your “Starter” policy

This contains:

- a) Policy statements and rules to confine the process.
- b) Permissions you want to give your program (consider e.g., files it needs to access, and how to give minimal permissions).

Let's look at a hello.te “starter” policy

Important point about my sample “Hello World” example starter module

It uses ‘attributes’. This is important to use (for your own sake).

If you don’t use them, it’s still possible to write your policy, but it will take more time.

What are attributes?

- Allows grouping of types in the policy
- Example you will see here: the domain attribute. Allows targeted policy to grant common privileges for domains by default.

Step 4: Get set up and run your “Starter” policy

- a) Compile and install your policy
- ~~b) label the required files with the new types~~
- ~~c) attempt to run it~~

Uh-oh... it didn't work!

Why not? Because your program needs many more permissions than you expect.

How do we figure out what it needs?

- The audit log

Audit2allow: use it, but with caution

It's very helpful. But it has some annoying behavior. Here's how to manage it:

- 1) ~~Remain aware that all it's doing is making the SELinux audit logs far more readable (and useful).~~
- 2) The audit logs contain errors from the entire system! You can wind up with stuff relating to e.g., your own system queries, other programs, etc. Tip: before you run audit2allow, rotate your audit log:

sudo service auditd rotate

- 3) ~~Be aware that audit logs will only contain errors if the error was triggered! This means ***all code paths must be executed during testing***. It also means that if an error halts the program early, it won't collect more logs.~~

Running audit2allow

Basic usage:

```
sudo more /var/log/audit/audit.log | audit2allow
```

If you only care about some logs, you can use grep!

Important things to consider:

- 1) Why should you run it?
- 2) When should you run it?
- 3) Will it generate a policy for you?
- 4) Will it generate a policy for you that takes care of confining the process to a new domain?
- 5) Will it work at all if the process is not confined yet?

Step 5: Use audit2allow to get your policy to work

- a) Run audit2allow and add the new rules
- b) You need to update your require block accordingly to ensure all types, classes, permissions, etc. are declared.
- c) Remove the old module, recompile, and reinstall

```
sudo semodule -r mymodule
```
- d) Re-run: did it work? If not, continue from a)

Getting it all to work: demo

Tip: Don't forget that if you update your policy, you should remove the old one before installing the new one.



Hello Secret Example

See `hellosecret.c`

Starting with step 1, what's different here? What should we do differently in steps 2 and 3?



Tip for using semodule fcontext

**** Make sure you use the **full path** of the file-name or directory-name**

1. `/usr/sbin/semanage fcontext -a options file-name|directory-name`

- E.g.: `/usr/sbin/semanage fcontext-a -t samba_share_t /etc/file1` changes file1's type to `samba_share_t`.
- The `-a` option adds a new record
- The `-t` option defines a type (`samba_share_t`).

2. `/sbin/restorecon -v file-name|directory-name`

- This applies the context changes